



UNIVERSIDADE DO VALE DO ITAJAÍ
CIÊNCIA DA COMPUTAÇÃO – PROJETO DE SISTEMAS DIGITAIS
PROF. PhD. DOUGLAS ROSSI DE MELO

GUSTAVO HENRIQUE STAHL MÜLLER

PROJETO DE CIRCUITOS SEQUENCIAIS

ITAJAÍ
2022

SUMÁRIO

1. INTRODUÇÃO.....	3
2. DESENVOLVIMENTO.....	4
2.1. PRIMEIRA MÁQUINA DE ESTADOS.....	4
2.1.1 IMPLEMENTAÇÃO	4
2.1.2 SIMULAÇÃO E DIAGRAMA RTL	6
2.2. SEGUNDA MÁQUINA DE ESTADOS	8
2.2.1 IMPLEMENTAÇÃO	8
2.2.2 SIMULAÇÃO E DIAGRAMA RTL	11
2.3. BLOCO DE CONTROLE DA PRIMEIRA MÁQUINA DE ESTADOS.....	13
2.3.1 ETAPA 1: CAPTURAR A MÁQUINA DE ESTADOS	13
2.3.2 ETAPA 2: CRIAR A ARQUITETURA.....	13
2.3.3 ETAPA 3: CODIFICAR OS ESTADOS.....	14
2.3.4 ETAPA 4: CRIAR A TABELA DE ESTADOS.....	14
2.3.5 ETAPA 5: IMPLEMENTAR A LÓGICA COMBINACIONAL	14
2.3.6 IMPLEMENTAÇÃO	15
2.3.7 SIMULAÇÃO E DIAGRAMA RTL	17

1. INTRODUÇÃO

Esse trabalho tem como objetivo consolidar os conhecimentos de projeto de circuitos sequenciais com o uso de VHDL, Quartus Prime e Questa, abrangendo a criação de três máquinas de estado.

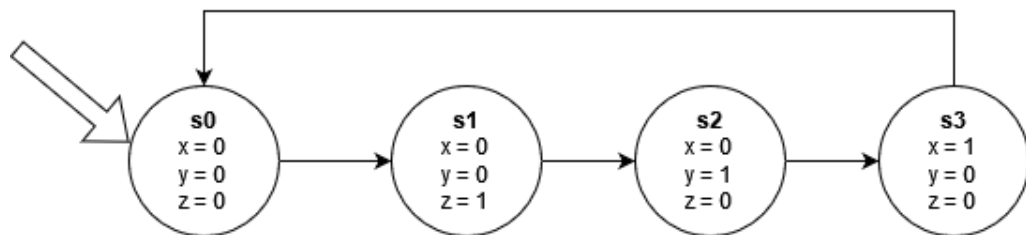
As duas primeiras usam processos explícitos e uso da cláusula *when* para definir as saídas e próximos estados, enquanto a última usa expressões booleanas e foi construída de acordo com as cinco etapas para a criação de blocos de controle, descritas no livro “Digital Design with RTL Design, VHDL, and Verilog”, de Frank Vahid.

2. DESENVOLVIMENTO

2.1. PRIMEIRA MÁQUINA DE ESTADOS

A primeira máquina de estados consiste de nenhuma entrada de dados e três saídas (chamadas de x, y e z) que alternam entre os valores 000, 001, 010 e 100. O valor 000 é o estado inicial da máquina e as transições ocorrem à cada subida da borda do relógio. Em todas as máquinas de estado desse trabalho, a condição de subida da borda do relógio é implicitamente presente em todas as transições.

Figura 1 – Diagrama de transições da primeira máquina de estados.



Fonte: O autor.

2.1.1 IMPLEMENTAÇÃO

Após definir o comportamento esperado do circuito através da máquina de estados, é possível codificar a entidade, arquitetura e testbench:

Figura 2 – Entidade e arquitetura da primeira máquina de estados.

```

library ieee;
use ieee.std_logic_1164.all;

entity fsm_1 is
  port
  (
    i_CLK: in std_logic;
    o_X: out std_logic;
    o_Y: out std_logic;
    o_Z: out std_logic
  );
end fsm_1;

architecture arch_fsm_1 of fsm_1 is
  type t_STATE is (s0, s1, s2, s3);
  signal r_STATE: t_STATE;
  signal w_NEXT_STATE: t_STATE;

  begin
    -- State register
    p_CHANGE_STATE: process(i_CLK) begin
      if (rising_edge(i_CLK)) then
        r_STATE <= w_NEXT_STATE;
      end if;
    end process;
  end arch_fsm_1;

```

```

        end if;
    end process;

    -- Combinational logic
    p_NEXT_STATE: process(r_STATE) begin
        case (r_STATE) is
            when s0 => w_NEXT_STATE <= s1;
            when s1 => w_NEXT_STATE <= s2;
            when s2 => w_NEXT_STATE <= s3;
            when s3 => w_NEXT_STATE <= s0;
            when others => w_NEXT_STATE <= s0;
        end case;
    end process;

    p_OUTPUT: process(r_STATE) begin
        case (r_STATE) is
            when s0 => o_X <= '0'; o_Y <= '0'; o_Z <= '0';
            when s1 => o_X <= '0'; o_Y <= '0'; o_Z <= '1';
            when s2 => o_X <= '0'; o_Y <= '1'; o_Z <= '0';
            when s3 => o_X <= '1'; o_Y <= '0'; o_Z <= '0';
            when others => o_X <= '0'; o_Y <= '0'; o_Z <= '0';
        end case;
    end process;
end arch_fsm_1;

```

Fonte: O autor.

Figura 3 – Testbench da primeira máquina de estados.

```

library ieee;
use ieee.std_logic_1164.all;

entity fsm_1_tb is
end fsm_1_tb;

architecture arch_fsm_1_tb of fsm_1_tb is
    component fsm_1 is
        port
        (
            i_CLK: in std_logic;
            o_X: out std_logic;
            o_Y: out std_logic;
            o_Z: out std_logic
        );
    end component;

    signal w_CLK: std_logic;
    signal w_X: std_logic;
    signal w_Y: std_logic;
    signal w_Z: std_logic;

    begin

        fsm_instance: fsm_1 port map (w_CLK, w_X, w_Y, w_Z);

        process begin
            -- s0
            w_CLK <= '0';
            wait for 1 ns;

```

```

assert (w_X = '0') report "Fail x @ 0" severity error;
assert (w_Y = '0') report "Fail y @ 0" severity error;
assert (w_Z = '0') report "Fail z @ 0" severity error;

-- s1
w_CLK <= '1';
wait for 1 ns;
w_CLK <= '0';
wait for 1 ns;

assert (w_X = '0') report "Fail x @ 1" severity error;
assert (w_Y = '0') report "Fail y @ 1" severity error;
assert (w_Z = '1') report "Fail z @ 1" severity error;

-- s2
w_CLK <= '1';
wait for 1 ns;
w_CLK <= '0';
wait for 1 ns;

assert (w_X = '0') report "Fail x @ 2" severity error;
assert (w_Y = '1') report "Fail y @ 2" severity error;
assert (w_Z = '0') report "Fail z @ 2" severity error;

-- s3
w_CLK <= '1';
wait for 1 ns;
w_CLK <= '0';
wait for 1 ns;

assert (w_X = '1') report "Fail x @ 3" severity error;
assert (w_Y = '0') report "Fail y @ 3" severity error;
assert (w_Z = '0') report "Fail z @ 3" severity error;

-- back to s0
w_CLK <= '1';
wait for 1 ns;
w_CLK <= '0';
wait for 1 ns;

assert (w_X = '0') report "Fail x @ 4" severity error;
assert (w_Y = '0') report "Fail y @ 4" severity error;
assert (w_Z = '0') report "Fail z @ 4" severity error;

wait;
end process;
end arch_fsm_1_tb;

```

Fonte: O autor.

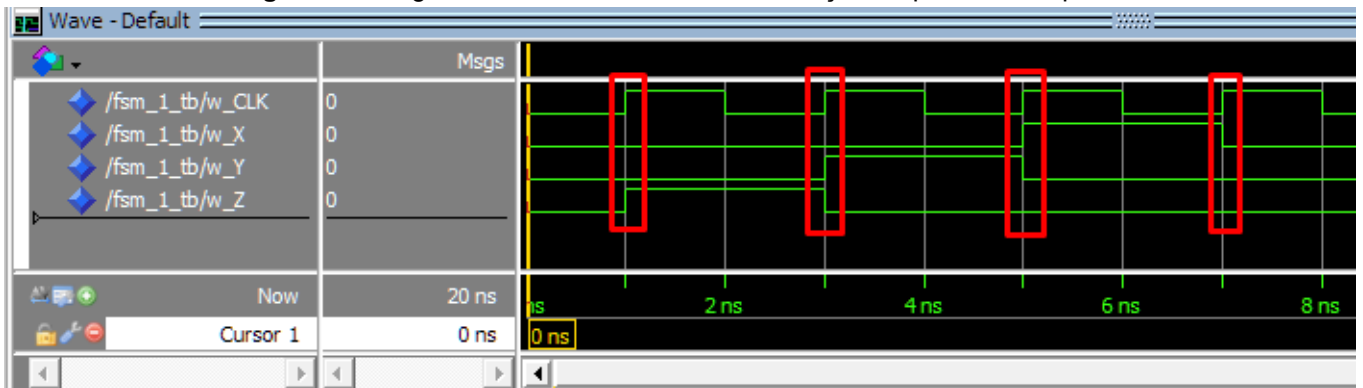
2.1.2 SIMULAÇÃO E DIAGRAMA RTL

Pelo diagrama de forma de onda, é possível concluir que o circuito se comporta corretamente:

- Inicialmente, x, y e z são iguais à 0;
- Quando o relógio sobe de 0 para 1 pela primeira vez, somente z é 1, e ele se mantém em 1 até a próxima subida do relógio;
- Quando o relógio sobe de 0 para 1 pela segunda vez, somente y é 1, e ele se mantém em 1 até a próxima subida do relógio;

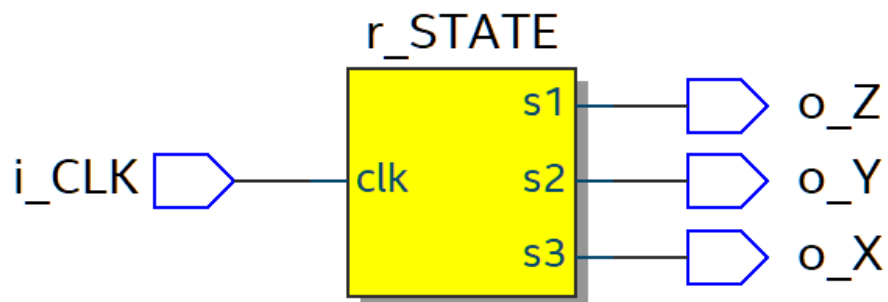
- Quando o relógio sobe de 0 para 1 pela terceira vez, somente x é 1, e ele se mantém em 1 até a próxima subida do relógio;
- Quando o relógio sobe de 0 para 1 pela quarta vez, todos os valores de saída são restaurados para 0, voltando para o estado inicial.

Figura 4 – Diagrama de forma de onda da simulação da primeira máquina de estados.



Fonte: O autor.

Figura 5 – Diagrama RTL da primeira máquina de estados.

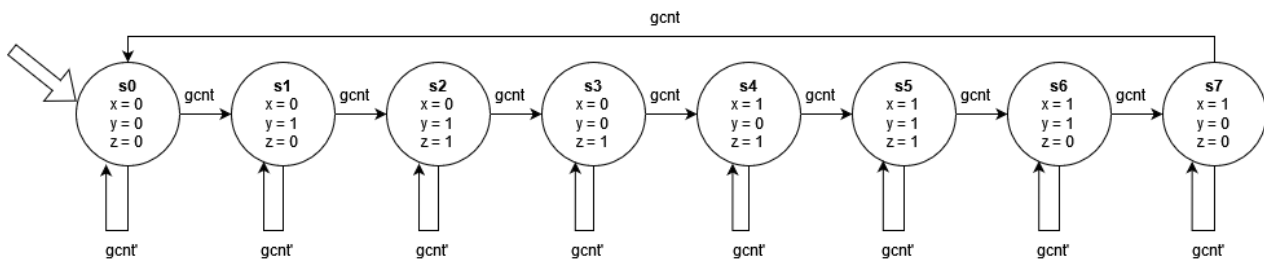


Fonte: O autor.

2.2. SEGUNDA MÁQUINA DE ESTADOS

A segunda máquina de estados consiste em uma entrada chamada de *gcnt* e três saídas (chamadas de *x*, *y* e *z*). As saídas geram a sequência do código Gray, onde somente uma das saídas é modificada de 1 para 0 ou de 0 para 1 em cada um dos valores. A sequência é 000, 010, 011, 001, 101, 111, 110 e 100. A saída somente muda em subidas do relógio e quando *gcnt* é igual à 1. O valor 000 é o estado inicial.

Figura 6 – Diagrama de transições da segunda máquina de estados.



Fonte: O autor.

2.2.1 IMPLEMENTAÇÃO

Após definir o comportamento esperado do circuito através da máquina de estados, é possível codificar a entidade, arquitetura e testbench:

Figura 7 – Entidade e arquitetura da segunda máquina de estados.

```

library ieee;
use ieee.std_logic_1164.all;

entity fsm_2 is
    port
    (
        i_CLK: in std_logic;
        i_GCNT: in std_logic;
        o_X: out std_logic;
        o_Y: out std_logic;
        o_Z: out std_logic
    );
end fsm_2;

architecture arch_fsm_2 of fsm_2 is
    type t_STATE is (s0, s1, s2, s3, s4, s5, s6, s7);
    signal r_STATE: t_STATE;
    signal w_NEXT_STATE: t_STATE;

    begin
        -- State register
        p_CHANGE_STATE: process(i_CLK) begin
            if (rising_edge(i_CLK) and i_GCNT = '1') then

```



```

        r_STATE <= w_NEXT_STATE;
    end if;
end process;

-- Combinational logic
p_NEXT_STATE: process(r_STATE) begin
    case (r_STATE) is
        when s0 => w_NEXT_STATE <= s1;
        when s1 => w_NEXT_STATE <= s2;
        when s2 => w_NEXT_STATE <= s3;
        when s3 => w_NEXT_STATE <= s4;
        when s4 => w_NEXT_STATE <= s5;
        when s5 => w_NEXT_STATE <= s6;
        when s6 => w_NEXT_STATE <= s7;
        when s7 => w_NEXT_STATE <= s0;
        when others => w_NEXT_STATE <= s0;
    end case;
end process;

p_OUTPUT: process(r_STATE) begin
    case (r_STATE) is
        when s0 => o_X <= '0'; o_Y <= '0'; o_Z <= '0';
        when s1 => o_X <= '0'; o_Y <= '1'; o_Z <= '0';
        when s2 => o_X <= '0'; o_Y <= '1'; o_Z <= '1';
        when s3 => o_X <= '0'; o_Y <= '0'; o_Z <= '1';
        when s4 => o_X <= '1'; o_Y <= '0'; o_Z <= '1';
        when s5 => o_X <= '1'; o_Y <= '1'; o_Z <= '1';
        when s6 => o_X <= '1'; o_Y <= '1'; o_Z <= '0';
        when s7 => o_X <= '1'; o_Y <= '0'; o_Z <= '0';
        when others => o_X <= '0'; o_Y <= '0'; o_Z <= '0';
    end case;
end process;
end arch_fsm_2;

```

Fonte: O autor.

Figura 8 – Testbench da segunda máquina de estados.

```

library ieee;
use ieee.std_logic_1164.all;

entity fsm_2_tb is
end fsm_2_tb;

architecture arch_fsm_2_tb of fsm_2_tb is
    component fsm_2 is
        port
        (
            i_CLK: in std_logic;
            i_GCNT: in std_logic;
            o_X: out std_logic;
            o_Y: out std_logic;
            o_Z: out std_logic
        );
    end component;

    signal w_CLK: std_logic;

```

```

signal w_GCNT: std_logic;
signal w_X: std_logic;
signal w_Y: std_logic;
signal w_Z: std_logic;

begin
    fsm_instance: fsm_2 port map (w_CLK, w_GCNT, w_X, w_Y, w_Z);

    process begin
        w_GCNT <= '1';

        -- s0
        w_CLK <= '0';
        wait for 1 ns;

        assert (w_X = '0') report "Fail x @ 0" severity error;
        assert (w_Y = '0') report "Fail y @ 0" severity error;
        assert (w_Z = '0') report "Fail z @ 0" severity error;

        -- s1
        w_CLK <= '1';
        wait for 1 ns;
        w_CLK <= '0';
        wait for 1 ns;

        assert (w_X = '0') report "Fail x @ 1" severity error;
        assert (w_Y = '1') report "Fail y @ 1" severity error;
        assert (w_Z = '0') report "Fail z @ 1" severity error;

        -- s2
        w_CLK <= '1';
        wait for 1 ns;
        w_CLK <= '0';
        wait for 1 ns;

        assert (w_X = '0') report "Fail x @ 2" severity error;
        assert (w_Y = '1') report "Fail y @ 2" severity error;
        assert (w_Z = '1') report "Fail z @ 2" severity error;

        -- s3
        w_CLK <= '1';
        wait for 1 ns;
        w_CLK <= '0';
        wait for 1 ns;

        assert (w_X = '0') report "Fail x @ 3" severity error;
        assert (w_Y = '0') report "Fail y @ 3" severity error;
        assert (w_Z = '1') report "Fail z @ 3" severity error;

        -- s4
        w_CLK <= '1';
        wait for 1 ns;
        w_CLK <= '0';
        wait for 1 ns;

        assert (w_X = '1') report "Fail x @ 4" severity error;
        assert (w_Y = '0') report "Fail y @ 4" severity error;
        assert (w_Z = '1') report "Fail z @ 4" severity error;

        -- s5
        w_CLK <= '1';
        wait for 1 ns;
        w_CLK <= '0';
        wait for 1 ns;
    end process;
end begin;

```

```

        assert (w_X = '1') report "Fail x @ 5" severity error;
        assert (w_Y = '1') report "Fail y @ 5" severity error;
        assert (w_Z = '1') report "Fail z @ 5" severity error;

        -- s6
        w_CLK <= '1';
        wait for 1 ns;
        w_CLK <= '0';
        wait for 1 ns;

        assert (w_X = '1') report "Fail x @ 6" severity error;
        assert (w_Y = '1') report "Fail y @ 6" severity error;
        assert (w_Z = '0') report "Fail z @ 6" severity error;

        -- s7
        w_CLK <= '1';
        wait for 1 ns;
        w_CLK <= '0';
        wait for 1 ns;

        assert (w_X = '1') report "Fail x @ 7" severity error;
        assert (w_Y = '0') report "Fail y @ 7" severity error;
        assert (w_Z = '0') report "Fail z @ 7" severity error;

        -- back to s0
        w_CLK <= '1';
        wait for 1 ns;
        w_CLK <= '0';
        wait for 1 ns;

        assert (w_X = '0') report "Fail x @ 8" severity error;
        assert (w_Y = '0') report "Fail y @ 8" severity error;
        assert (w_Z = '0') report "Fail z @ 8" severity error;

        -- gcnt equals '0': should not change state even on clock rise
        w_GCNT <= '0';
        w_CLK <= '1';
        wait for 1 ns;
        w_CLK <= '0';
        wait for 1 ns;

        assert (w_X = '0') report "Fail x @ 9" severity error;
        assert (w_Y = '0') report "Fail y @ 9" severity error;
        assert (w_Z = '0') report "Fail z @ 9" severity error;

        wait;
    end process;

end arch_fsm_2_tb;

```

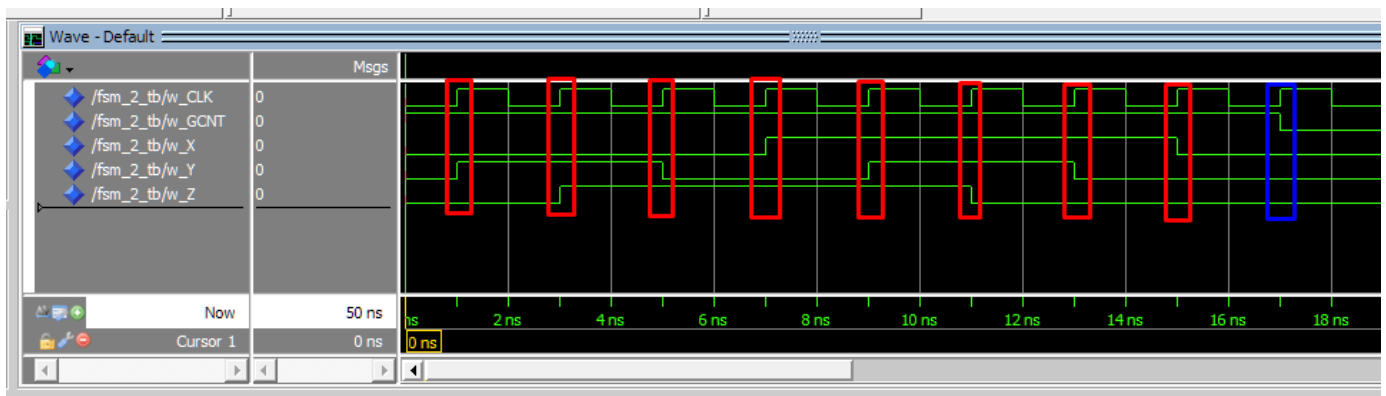
Fonte: O autor.

2.2.2 SIMULAÇÃO E DIAGRAMA RTL

Na simulação, as bordas de subida do relógio são circuladas em vermelho. Como *gcnt* é igual à 1 em todas as bordas de subida do relógio menos a última, as primeiro 8 subidas do relógio representam transições entre estados diferentes que alternam os valores de *x*, *y* e *z*.

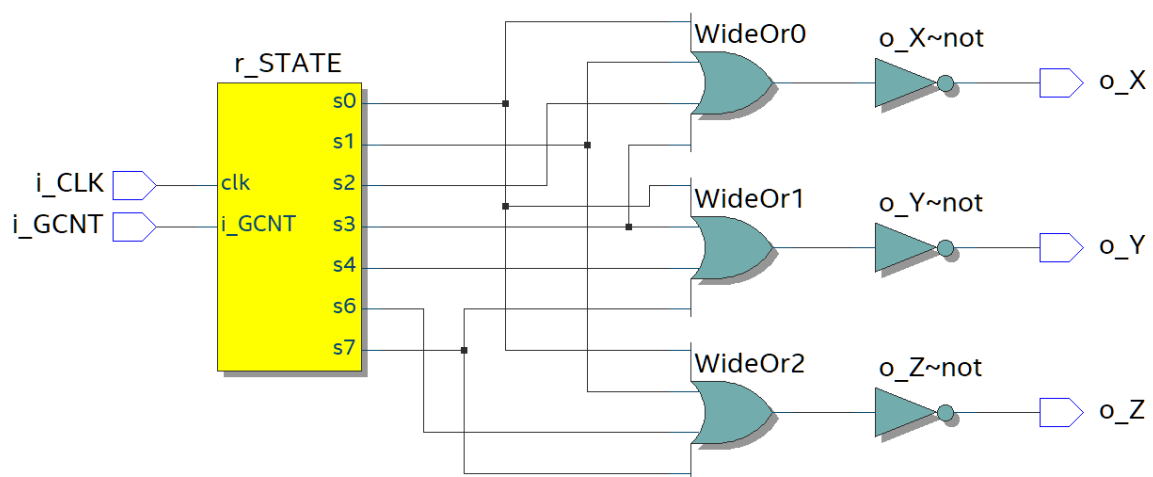
- Inicialmente, x , y e z são iguais à 0, e $gcnt$ é 1;
- Na subida de relógio 1 (instante 1 ns), a saída xyz se torna 010;
- Na subida de relógio 2 (instante 3 ns), a saída xyz se torna 011;
- Na subida de relógio 3 (instante 5 ns), a saída xyz se torna 001;
- Na subida de relógio 4 (instante 7 ns), a saída xyz se torna 101;
- Na subida de relógio 5 (instante 9 ns), a saída xyz se torna 111;
- Na subida de relógio 6 (instante 11 ns), a saída xyz se torna 110;
- Na subida de relógio 7 (instante 13 ns), a saída xyz se torna 100;
- Na subida de relógio 8 (instante 15 ns), a saída xyz se torna 000, indicando que a máquina voltou para o estado inicial.
- Na subida de relógio 9 (circulada em azul no instante 17 ns), as saídas permanecem em 000, já que a máquina ainda está no estado inicial pois $gcnt$ se tornou 0 no momento anterior.

Figura 9 – Diagrama de forma de onda da simulação da segunda máquina de estados.



Fonte: O autor.

Figura 10 – Diagrama RTL da segunda máquina de estados.



Fonte: O autor.

2.3. BLOCO DE CONTROLE DA PRIMEIRA MÁQUINA DE ESTADOS

O bloco de controle corresponde a primeira máquina de estados e foi implementado de acordo com as cinco etapas do processo de bloco de controle descritas no livro de Frank Vahid:

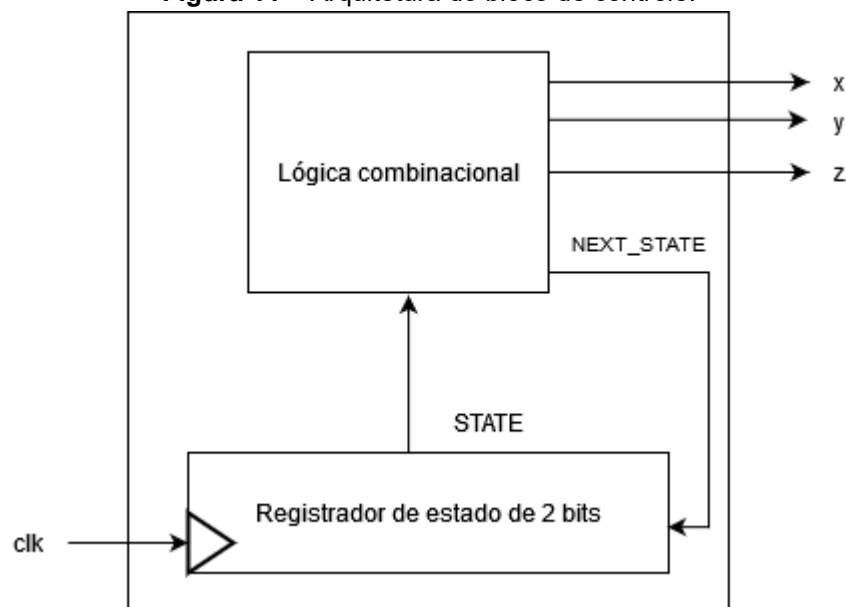
2.3.1 ETAPA 1: CAPTURAR A MÁQUINA DE ESTADOS

A primeira etapa é capturar a máquina de estados que o bloco de controle implementa. Neste caso, a máquina de estados que será implementada foi descrita na seção 2.1.

2.3.2 ETAPA 2: CRIAR A ARQUITETURA

A segunda etapa é criar a arquitetura. Neste caso, ela não possui nenhuma entrada de dados externa e possui três saídas (x, y, e z). Internamente, a lógica combinacional possui como entrada o estado atual (representado por 2 bits) e possui como saída o próximo estado (representado por 2 bits) e os valores x, y e z.

Figura 11 – Arquitetura do bloco de controle.



Fonte: O autor.

2.3.3 ETAPA 3: CODIFICAR OS ESTADOS

A terceira etapa é codificar os estados. Como a máquina possui 4 estados, o tamanho mínimo necessário para o registrador de estado é 2 bits.

Figura 12 – Codificação dos estados do bloco de controle.

Estado	Codificação
s0	00
s1	01
s2	10
s3	11

Fonte: O autor.

2.3.4 ETAPA 4: CRIAR A TABELA DE ESTADOS

A quarta etapa é criar a tabela de estados, que é uma tabela-verdade que mapeia as entradas possíveis (incluindo os bits de estado) com as saídas possíveis (incluindo os bits do próximo estado).

Os bits do estado atual são representados por SB(0) e SB(1) (state bit 0 e state bit 1, respectivamente) e os bits do próximo estado são representados por NS(0) e NS(1) (next state 0 e next state 1, respectivamente).

Figura 13 – Tabela de estados do bloco de controle.

SB(1)	SB(0)	x	y	z	NS(1)	NS(0)
0	0	0	0	0	0	1
0	1	0	0	1	1	0
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Fonte: O autor.

2.3.5 ETAPA 5: IMPLEMENTAR A LÓGICA COMBINACIONAL

A lógica combinacional foi implementada com o uso de equações booleanas que foram derivadas da tabela de estados da etapa passada:

Figura 14 – Equações booleanas da lógica combinacional do bloco de controle.

$X = \mathbf{SB(1) AND SB(0)};$ $Y = \mathbf{SB(1) AND NOT SB(0)};$ $Z = \mathbf{NOT SB(1) AND SB(0)};$ $NS(1) = (\mathbf{NOT SB(1) AND SB(0)}) \mathbf{OR (SB(1) AND NOT SB(0))};$ $NS(0) = (\mathbf{NOT SB(1) AND NOT SB(0)}) \mathbf{OR (SB(1) AND NOT SB(0))};$

Fonte: O autor.

2.3.6 IMPLEMENTAÇÃO

Após definir o comportamento esperado do circuito através da tabela verdade e a sua implementação através das equações booleanas, é possível codificar a entidade, arquitetura e testbench:

Figura 15 – Entidade e arquitetura do bloco de controle.

```
library ieee;
use ieee.std_logic_1164.all;

entity fsm_3 is
    port
    (
        i_CLK: in std_logic;
        o_X: out std_logic;
        o_Y: out std_logic;
        o_Z: out std_logic
    );
end fsm_3;

architecture arch_fsm_3 of fsm_3 is
    signal r_STATE: std_logic_vector (1 downto 0) := "00";
    signal w_NEXT_STATE: std_logic_vector (1 downto 0);

    begin
        -- State register
        p_CHANGE_STATE: process(i_CLK) begin
            if (rising_edge(i_CLK)) then
                r_STATE <= w_NEXT_STATE;
            end if;
        end process;

        -- Combinational logic

        -- Next state
        p_NEXT_STATE: process(r_STATE) begin
            w_NEXT_STATE(1) <= (not r_STATE(1) and r_STATE(0)) or (r_STATE(1) and
not r_STATE(0));
            w_NEXT_STATE(0) <= (not r_STATE(1) and not r_STATE(0)) or (r_STATE(1)
and not r_STATE(0));
        end process;

        -- Output
        o_X <= r_STATE(1) and r_STATE(0);
        o_Y <= r_STATE(1) and not r_STATE(0);
        o_Z <= not r_STATE(1) and r_STATE(0);
    end arch_fsm_3;
```

Fonte: O autor.

Figura 16 – Testbench do bloco de controle.

```
library ieee;
use ieee.std_logic_1164.all;

entity fsm_3_tb is
end fsm_3_tb;
```

```

architecture arch_fsm_3_tb of fsm_3_tb is
  component fsm_3 is
    port
    (
      i_CLK: in std_logic;
      o_X: out std_logic;
      o_Y: out std_logic;
      o_Z: out std_logic
    );
  end component;

  signal w_CLK: std_logic;
  signal w_X: std_logic;
  signal w_Y: std_logic;
  signal w_Z: std_logic;

begin
  fsm_instance: fsm_3 port map (w_CLK, w_X, w_Y, w_Z);

  process begin
    -- s0
    w_CLK <= '0';
    wait for 1 ns;

    assert (w_X = '0') report "Fail x @ 0" severity error;
    assert (w_Y = '0') report "Fail y @ 0" severity error;
    assert (w_Z = '0') report "Fail z @ 0" severity error;

    -- s1
    w_CLK <= '1';
    wait for 1 ns;
    w_CLK <= '0';
    wait for 1 ns;

    assert (w_X = '0') report "Fail x @ 1" severity error;
    assert (w_Y = '0') report "Fail y @ 1" severity error;
    assert (w_Z = '1') report "Fail z @ 1" severity error;

    -- s2
    w_CLK <= '1';
    wait for 1 ns;
    w_CLK <= '0';
    wait for 1 ns;

    assert (w_X = '0') report "Fail x @ 2" severity error;
    assert (w_Y = '1') report "Fail y @ 2" severity error;
    assert (w_Z = '0') report "Fail z @ 2" severity error;

    -- s3
    w_CLK <= '1';
    wait for 1 ns;
    w_CLK <= '0';
    wait for 1 ns;

    assert (w_X = '1') report "Fail x @ 3" severity error;
    assert (w_Y = '0') report "Fail y @ 3" severity error;
    assert (w_Z = '0') report "Fail z @ 3" severity error;

    -- back to s0
    w_CLK <= '1';
    wait for 1 ns;
    w_CLK <= '0';
    wait for 1 ns;
  end process;
end architecture arch_fsm_3_tb;

```



```

        assert (w_X = '0') report "Fail x @ 4" severity error;
        assert (w_Y = '0') report "Fail y @ 4" severity error;
        assert (w_Z = '0') report "Fail z @ 4" severity error;

        wait;
    end process;
end arch_fsm_3_tb;

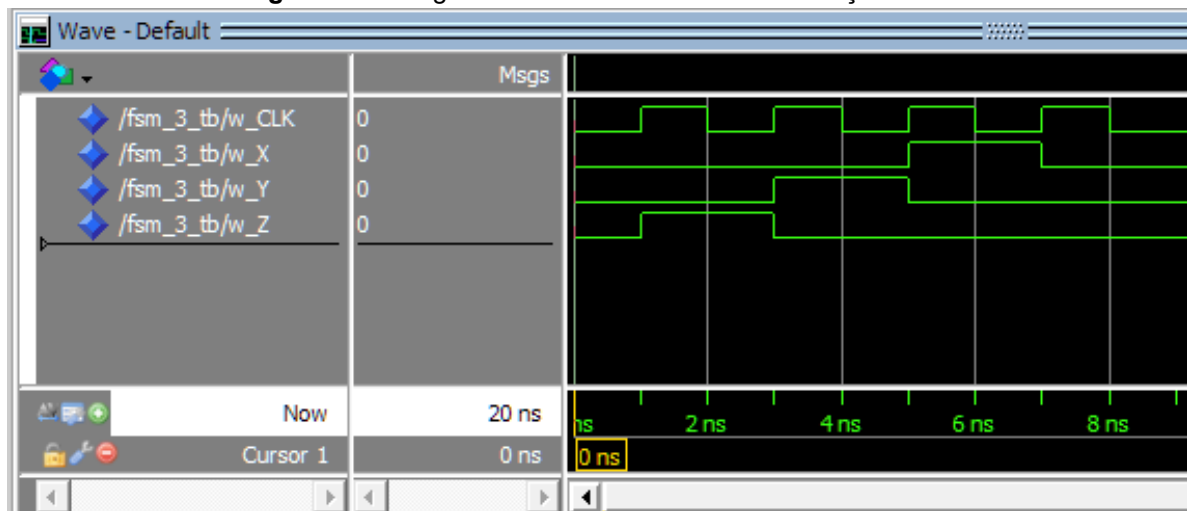
```

Fonte: O autor.

2.3.7 SIMULAÇÃO E DIAGRAMA RTL

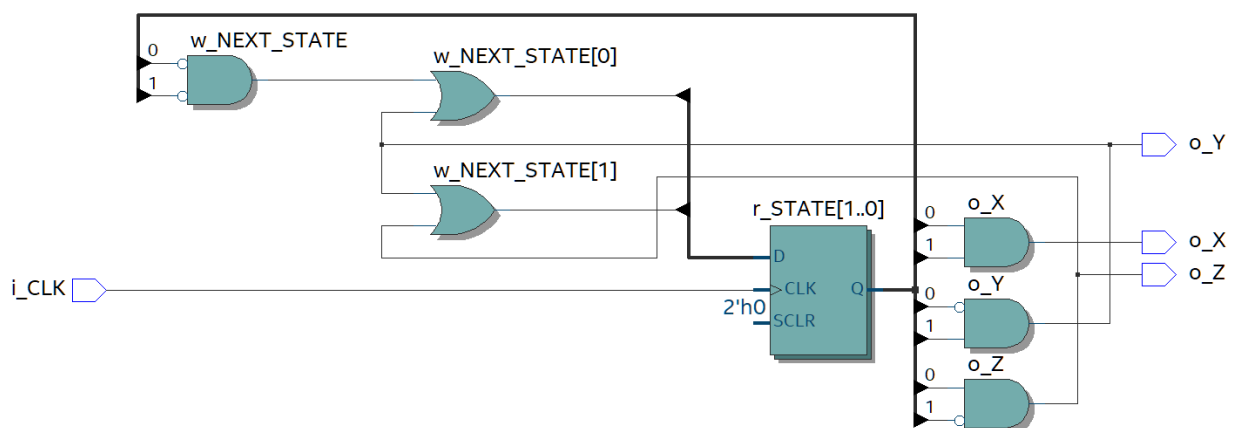
Pelo diagrama de forma de onda, é possível concluir que o circuito se comporta corretamente, pois o seu comportamento é idêntico ao comportamento da primeira máquina de estados.

Figura 17 – Diagrama de forma de onda da simulação do bloco de controle.



Fonte: O autor.

Figura 18 – Diagrama RTL do bloco de controle.



Fonte: O autor.